

Robust Image Classification Under Bandwidth Constraints

항공전자정보공학부

Donghyeon Kim
Korea Aerospace University

ID: 2019124018



한국항공대학교
Korea Aerospace University

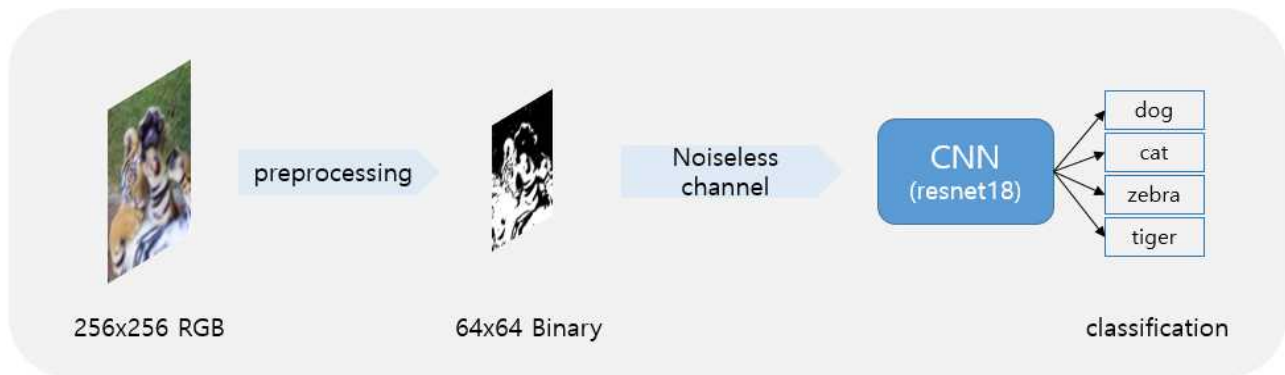


Figure 2. Overview of the image classification system under bandwidth constraints. A 256×256 RGB input image undergoes preprocessing to be binarized and resized into a 64×64 binary image, satisfying the 2^{12} -bit channel bandwidth limitation. This processed image is transmitted through a noiseless channel and fed into a CNN classifier (ResNet18), which outputs one of the predefined animal classes: dog, cat, zebra, or tiger.

Problem 1: Image preprocessing for classification, part 1.

- The objective is to design an image classification system for input images.
- However, as depicted in the figure, the image must be transmitted by a channel with a severe bandwidth restriction. The channel's bandwidth is limited to transmitting 2^{12} -bit data. For example, only data equivalent to a 64×64 binary image can pass this channel.
- Consequently, the original image should be processed to meet this bandwidth limit.

1) Design a CNN model to classify input color images. It will be upper-limitation of the system.

problem 1.1

2) Resize the image to 64×64, and binarize this image for the channel restriction. Then, design a CNN model to classify these binarized images. Compare the performance degradations.

problem 1.2

3) Apply your own binarization methods as preprocessing. Note that you can apply any types of image processing methods to maximize the classification performance.

problem 1.3

problem 1.1

[ResNet-18 for classification]

a) Residual Connections (Skip Connections)

A key feature of ResNet-18 is the use of residual connections, which allow the network to learn identity mappings by adding the input of a layer directly to its output. Formally, the residual block computes an output y as: $y = F(x) + x$ where x is the input and $F(x)$ denotes the output of a series of convolutional layers. This architecture facilitates the training of deeper networks by mitigating the vanishing gradient problem and improving gradient flow during backpropagation.

b) Layer Composition of ResNet-18

ResNet-18 consists of 18 layers, including convolutional, batch normalization, ReLU, and pooling operations. The architecture begins with a 7×7 convolution followed by max pooling, and then proceeds through four residual stages, each containing two residual blocks. After global average pooling, the output is passed through a fully connected layer for classification.

c) Customizing ResNet-18 for Image Classification

We customized the ResNet-18 model to perform 4-class image classification. We used 256×256 RGB images as input to establish the upper bound performance of the network.

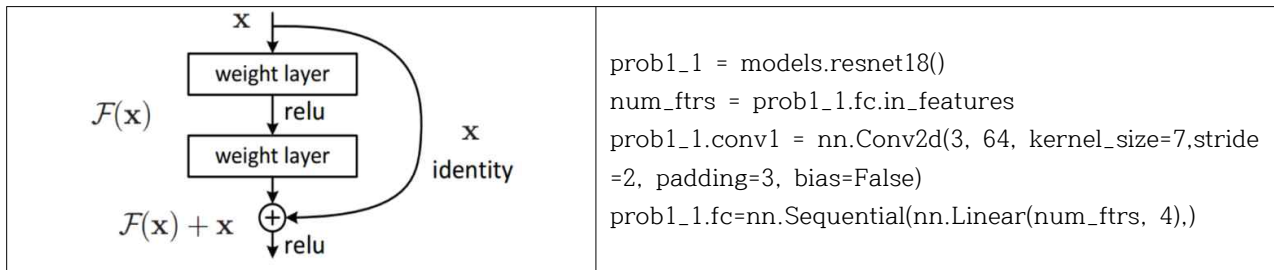


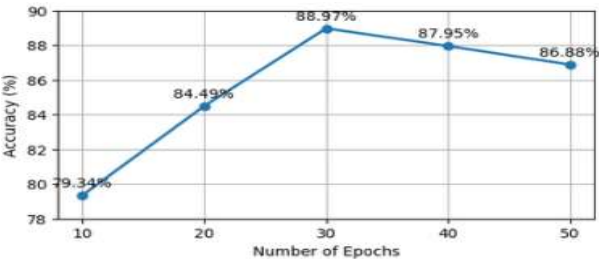
Figure 3. Illustration of a residual block structure (left) and the corresponding code for ResNet-18 model (right).

[Performance]

Table 1 shows the classification accuracy over training epochs. As the number of epochs increases, the accuracy improves significantly, reaching a peak of 88.97% at 30 epochs. After that point, the performance slightly decreases, indicating potential overfitting beyond 30 epochs. This suggests that optimal performance is achieved with moderate training duration.

Epochs	ACC(%)
10	79.34
20	84.49
30	88.97
40	87.95
50	86.88

Table 1. Classification Accuracy over Epochs



problem 1.2

[Image Transformation through Preprocessing]

- a) **Resize**
- Reducing the image resolution often results in the loss or distortion of fine-grained visual features, including small-scale objects, contours, textures, and detailed patterns.
- b) **Binarization**
- Binarization reduces the pixel-level information of an image to a single bit per pixel, which significantly lowers data size but can also lead to the loss of subtle visual details such as shading, gradients, and fine textures. In contrast to the 256×256 case, binarizing images at 64×64 resolution causes edge distortion and aliasing, primarily due to reduced spatial information.(Figure 4.)

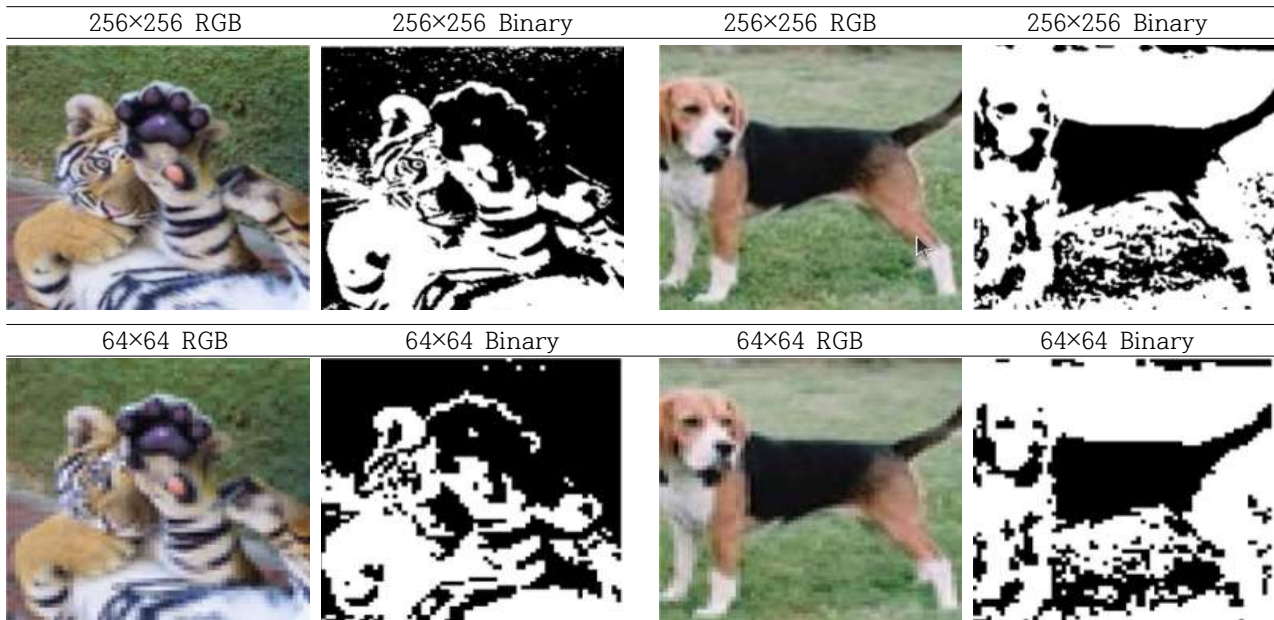


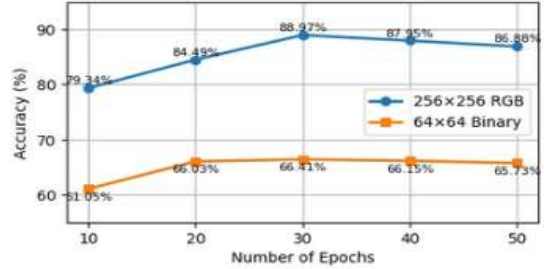
Figure 4. Image Transformation through Preprocessing

[Compare the performance degradations]

Table 2 illustrates the classification accuracy across training epochs for both high-resolution RGB images (256×256) and binarized low-resolution images (64×64). The RGB-based model shows a steady increase in accuracy, peaking at 88.97% at 30 epochs, followed by a slight decline likely due to overfitting. In contrast, the model trained on 64×64 binary images achieves its highest accuracy of 66.41% also at 30 epochs, but its performance remains relatively stable across epochs, with limited variation.

Epochs	ACC(%)	
	256×256 RGB	64×64 Binary
10	79.34	61.05
20	84.49	66.03
30	88.97	66.41
40	87.95	66.15
50	86.88	65.73

Table 2. Classification Accuracy over Epochs



[Analysis: Reasons of degradations]

a) Loss of Spatial Resolution

Downsampling the input image from 256×256 to 64×64 significantly reduces spatial resolution, resulting in the loss of small-scale details such as object contours, textures, and fine structures.

b) Binarization-Induced Information Loss

Binarization converts each pixel to a single bit, effectively removing all grayscale and color information. This transformation limits the image to coarse structural outlines while eliminating subtle visual cues such as gradients and shading, which are essential for discriminative learning in CNNs.

c) Edge Degradation and Aliasing Effects

The combination of resolution reduction and binarization leads to jagged edges and aliasing artifacts. These distortions can confuse the network, especially when distinguishing between objects with similar shapes but different visual textures or edge definitions.

problem 1.3

1. Problem Analysis

a) Downsampling

Type of Information	Explanation
Fine spatial detail	Small objects, thin lines, and textures disappear when the image is resized to a lower resolution.
Precise object boundaries	Smooth or complex contours may be simplified or distorted.
Small-scale features	High-frequency patterns or distant background elements are often lost entirely.

Table 3. Information Loss That Cannot Be Recovered

Type of Information	Possible?	Example Techniques
Contrast preservation	Yes	Bicubic interpolation
Anti-aliasing artifacts	Yes	Low-pass filtering before resizing

Table 4. Information Loss That Can Be Mitigated by Preprocessing

b) Binarization

Type of Information	Explanation
Color information	Completely discarded during grayscale conversion and binarization.
Grayscale gradients	Smooth transitions between brightness levels are collapsed.
Fine texture patterns	Subtle variations in pixel intensity are lost during transformation.

Table 5. Information Loss That Cannot Be Recovered

Type of Information	Possible?	Example Techniques
Edges extraction	Yes	Edge extraction before binarization
Contrast preservation	Yes	Adaptive thresholding, Otsu
Local texture extraction	Limited	Local Binary Patterns (LBP)

Table 6. Information Loss That Can Be Mitigated by Preprocessing

2. Approaches

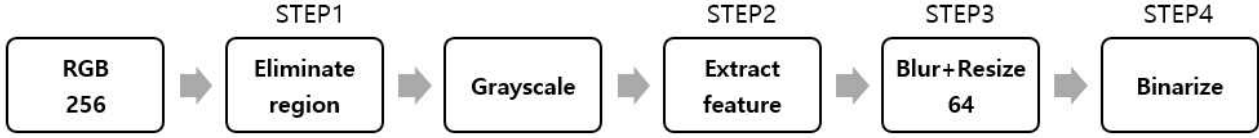


Figure 5. overview of our approaches

The problem is divided into four stages, and for each stage, we evaluate multiple techniques to identify the most effective one. The best-performing methods from each step are then integrated to form the overall classification pipeline.

Step1. Elimination of Non-informative Regions

- Assuming the object is located at the center of the image.
- Focus on the object by eliminating the surrounding borders or background.
- ex) Cropping, GrabCut

Step2. Feature extraction

- Binarization leads to significant loss of image information.
- Preserving fine details is unnecessary. (face and body share few common features).
- Redefine the problem as a simpler classification task by directly extracting the object's feature.
- ex) Laplacian of Gaussian, Sobel, Local Binary Patterns

Step3. Resizing

- Minimize the loss of information during the image downsampling process.
- Contrast preservation, Anti-aliasing artifacts
- ex) Gaussian blur, Bicubic interpolation

Step4. Binarization

- A binarization technique should be suitable for the extracted features.
- ex) Otsu, Adaptive thresholding

3. STEP1. Elimination of Non-informative Regions

a) Cropping: Based on the assumption that the object is centered, this approach removes the image borders. It offers the advantages of simplicity and fast computation.

b) GrabCut

Theoretical Motivation. Binary image segmentation algorithm that assigns each pixel in the input image a foreground/background label. It formulates the problem as a graph-based energy minimization task using statistical modeling via Gaussian Mixture Models (GMM). We also applied GrabCut under the assumption that the object is located near the center of the image.

Mechanism of GrabCut. GrabCut minimizes the following energy function

$$E(\alpha, \theta, z) = U(\alpha, \theta, z) + V(\alpha, z)$$

α : label vector indicating foreground or background for each pixel

θ : GMM parameters for foreground and background

z : RGB vector of pixel i

Unary Term U: Data Likelihood, Represents the likelihood that pixel z_i belongs to the assigned class α_i

$$U(\alpha, \theta, z) = \sum_i -\log P(z_i | \alpha_i, \theta)$$

Each class is modeled by a GMM with K Gaussian components

$$P(z_i | \alpha_i, \theta) = \sum_{k=1}^K \pi_k^{(\alpha_i)} \cdot N(z_i | \mu_k^{(\alpha_i)}, \Sigma_k^{(\alpha_i)})$$

π_k : mixture weight

μ_k, Σ_k : mean and covariance of the k-th Gaussian component

Pairwise Term V: Encourages adjacent pixels with similar colors to have the same label(Boundary preservation)

$$V(\alpha, z) = \gamma \sum_{(i,j) \in N} [\alpha_i \neq \alpha_j] \cdot \exp(-\beta \|z_i - z_j\|^2)$$

N : set of neighboring pixel pairs

$\beta = \frac{1}{2 \text{avg}(\|z_i - z_j\|^2)}$ is used to adapt the weight to image contrast

To minimize the energy, GrabCut constructs a graph with the following structure

- Nodes: Each pixel is a node. Two additional terminal nodes, Source(S) and Sink(T), represent foreground and background respectively
- Edges: T-link(connect each pixel to S and T with weights derived from unary term U), N-links(connect neighboring pixels with weights from pairwise term V)

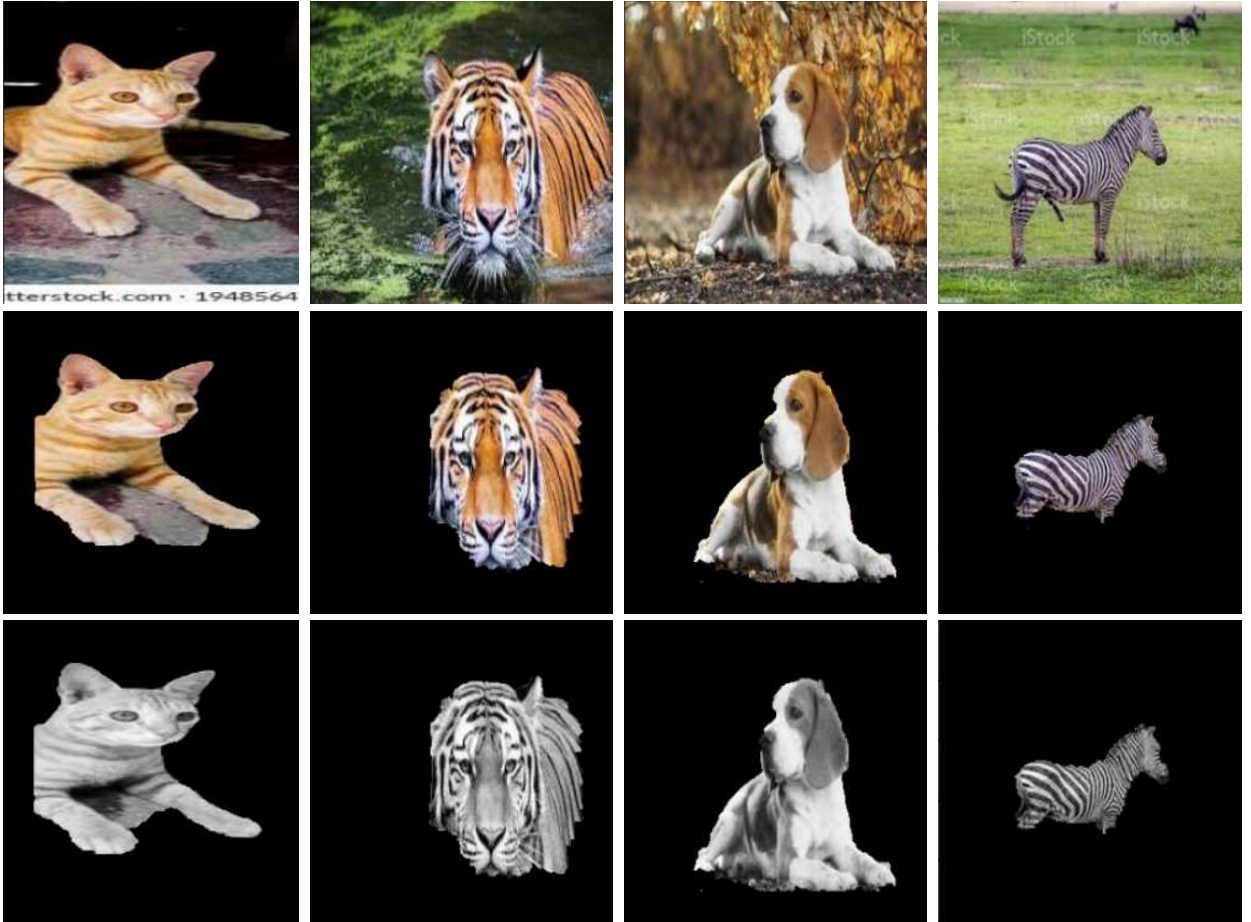


Figure 6. original images(top), Grabcut images(mid), grayscale after Grabcut(bottom)

As shown in Figure 6, the images processed with GrabCut retain only the target object while the background is almost completely removed. However, since the algorithm relies heavily on the initial rectangle, it may fail to distinguish between foreground and background correctly in certain images.



Figure 7. Limitations of Grabcut

In the left case, the object is too large compared to the initial bounding box, causing it to be unrecognized. While the right case, two objects present in the image, leading to a failure in detection.

Performance(crop vs Grabcut)

Epochs	ACC(%) on GrayScale(64×64)					
	baseline	Crop 1	Crop 2	Crop 3	Crop 4	GrabCut
10	68.42	68.97	70.23	71.05	69.47	64.36
20	69.91	70.13	71.24	71.32	70.98	68.49
30	70.02	71.84	76.71	72.24	73.26	70.25
40	70.14	75.39	77.50	73.03	72.37	70.44
50	70.26	74.89	77.14	73.29	70.16	70.32

Table 7. Classification Accuracy over Epochs with several methods

Through Table 7, we confirmed that images cropped by 2 pixels yield the best classification performance, and thus we adopt this setting in Step 1.

4. STEP2. Feature extraction

※ Based on the crop2 setting used in Step 1.

a) Edge extraction with LoG+Sobel

Theoretical Motivation. In classification tasks, the object's outline provides rich information—especially when the input image is binarized. Therefore, we extract edges from the image using a combination of the Laplacian of Gaussian (LoG) and Sobel filters. We first extract general edge structures using the Sobel filter, then reinforce the result by detecting finer edge details with the Laplacian of Gaussian (LoG) filter.

Mechanism of LOG. Second-order edge detection method that combines Gaussian smoothing with the Laplacian operator. It enhances regions of rapid intensity change (edges) while reducing noise through Gaussian filtering. The LoG operator is defined as the Laplacian of a Gaussian-filtered image:

$$LoG(x, y) = \nabla^2 [G(x, y) * I(x, y)]$$

In practice, the LoG operator highlights regions where the intensity changes sharply and is particularly effective in detecting fine edges. Its zero-crossings (where the output changes sign) are commonly used to locate edges.

Mechanism of Sobel. Discrete differentiation operator, used to compute an approximation of the gradient of the image intensity function. It applies two 3×3 convolution kernels to the image: one for detecting changes in the horizontal direction (G_x) and one for the vertical direction (G_y). The gradient magnitude at each pixel is then calculated as:

$$G(x, y) = \sqrt{G_x^2 + G_y^2}$$

where:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Performance

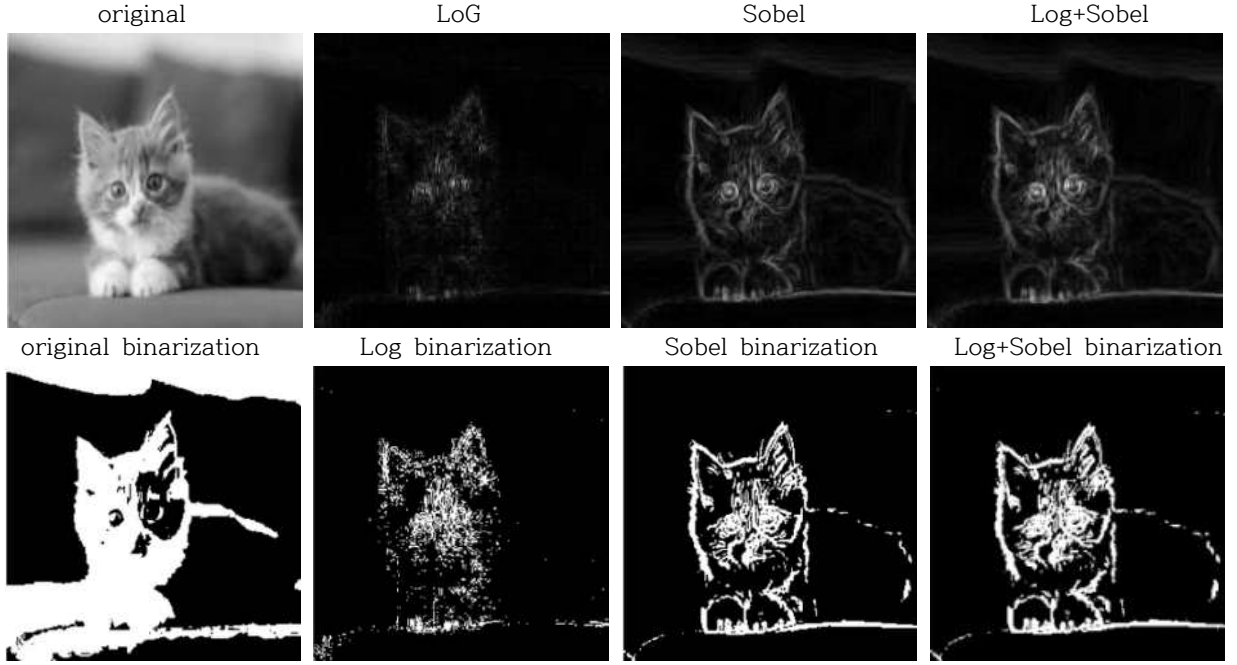


Figure 8. Preprocessing with LoG filter

Epochs	ACC(%) on Binary(64×64) with LoG		
	Baseline	LoG+Sobel	step1+LoG+Sobel
10	61.05	61.22	62.21
20	66.03	66.47	66.69
30	66.41	66.89	68.52
40	66.15	67.23	69.11
50	65.73	68.18	70.07
Max	66.41	68.18	70.07

Table 8. Classification Accuracy over Epochs with several preprocessing methods

b) Pattern extraction with Local Binary Patterns(LBP)

Theoretical Motivation. In image classification tasks, LBP can be employed either as a handcrafted feature extractor or as a preprocessing step that enhances textural patterns.

LBP is particularly effective at encoding micro-texture information such as fur, scales, wrinkles, or grain structures, which are critical in domains like medical imaging, material classification, and fine-grained animal recognition. And histograms of LBP codes can serve as compact and discriminative feature vectors, reducing the dimensionality of the input and mitigating overfitting in low-data regimes.

Mechanism of LBP. For a pixel I_c at position (x, y) , and its P circularly symmetric neighbors $[I_p]_{p=0}^{P-1}$ at radius R , the LBP code is computed as:

$$LBP_{P,R}(x, y) = \sum_{p=0}^{P-1} s(I_p - I_c) \cdot 2^p$$

the function $s()$ is defined as

$$s(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

The resulting LBP code represents the local binary pattern around a pixel, encoding texture by comparing the center pixel to its neighbors. These codes are often aggregated into histograms over small image regions to form compact feature vectors.

Performance

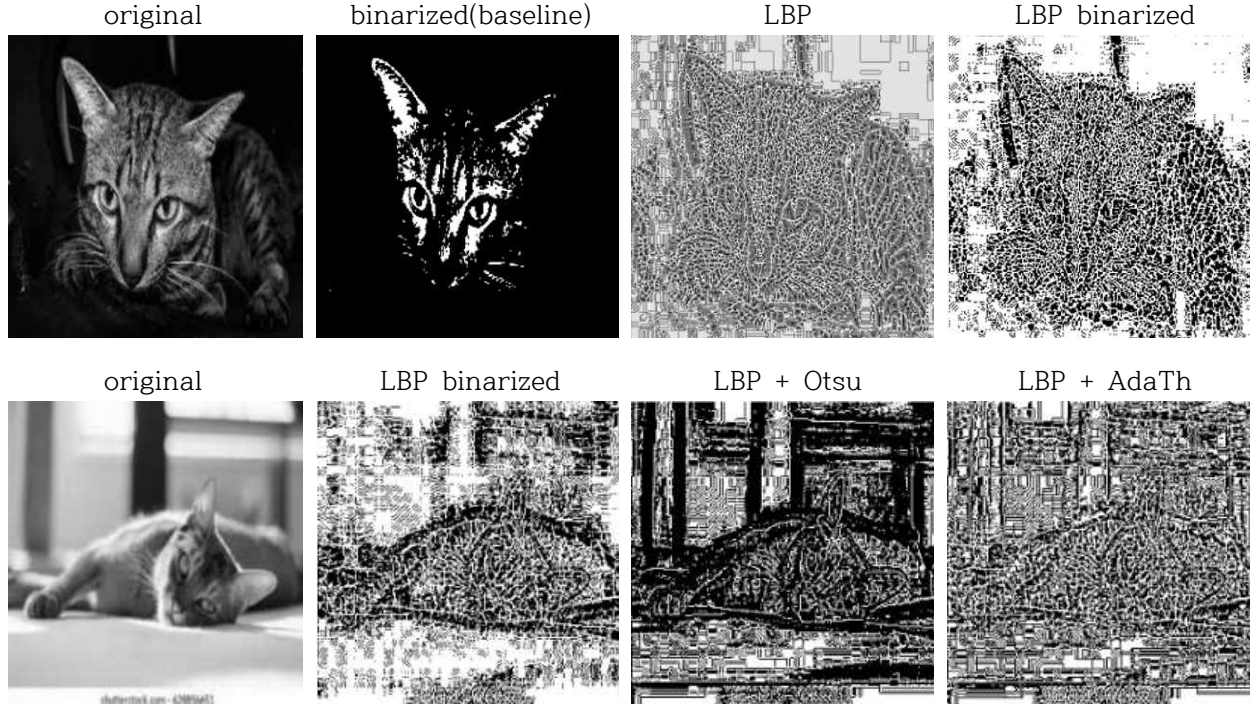


Figure 9. Preprocessing with LBP

Epochs	ACC(%) on Binary(64×64) with LBP		
	Baseline	LBP	step1+LBP
10	61.05	60.77	62.12
20	66.03	65.79	66.17
30	66.41	66.98	67.23
40	66.15	67.04	68.64
50	65.73	67.01	67.33
Max	66.41	67.04	68.64

Table 9. Classification Accuracy over Epochs with several preprocessing methods

Tables 9 and 10 demonstrate that the proposed method leads to improved performance. In particular, the combined use of LoG and Sobel achieved 70.07% accuracy, representing a 3.66% improvement over the baseline. Therefore, we adopt the LoG+Sobel approach in Step 2 for use in subsequent stages.

5. STEP3. Resizing

※ Based on Step 2.

During the downsampling process—i.e., when reducing the resolution—high-frequency components may be lost or distorted. To address this, it is necessary to apply appropriate filtering in advance to minimize information loss, or to use interpolation and post-processing techniques to restore the lost fine details.

a) Gaussian blur

Theoretical Motivation and Mechanism. Reducing high-frequency components with a Gaussian function. It is particularly significant in the context of image resizing, where it serves to suppress aliasing artifacts and improve visual smoothness. Gaussian blur is a type of low-pass filter that smooths an image by averaging pixel values in a local neighborhood, weighted by a Gaussian kernel. The 2D Gaussian function is defined as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Application in downsampling. When reducing the resolution of an image, aliasing artifacts occur due to undersampling of high-frequency components. According to the Nyquist-Shannon sampling theorem, a signal must be band-limited before sampling to avoid aliasing. In image processing, this is achieved by applying a low-pass filter prior to downsampling.

b) Bicubic interpolation

Theoretical Motivation and Mechanism. Resampling technique in image processing that provides smooth and high-quality image resizing by considering the intensity values of a 4×4 neighborhood of pixels.

For a given point (x, y) in the resized image, the intensity value is computed as a weighted sum of the 16 surrounding pixels in the original image.

$$I(x, y) = \sum_{i=-1}^2 \sum_{j=-1}^2 w(i, j) \cdot I(x_i + i, y_j + j)$$

Here, $w(i, j)$ represents the interpolation weights determined by a bicubic kernel, and $I(x_i + i, y_j + j)$ are the pixel intensities in the original image.

A commonly used bicubic kernel is the Keys cubic convolution kernel:

$$k(a) = \begin{cases} (a+2)|a|^3 - (a+3)|a|^2 + 1 & \text{for } |a| \leq 1 \\ a|a|^3 - 5a|a|^2 + 8a|a| - 4a & \text{for } 1 < |a| < 2 \\ 0 & \text{otherwise} \end{cases}$$

Application in downsampling. combined with a low pass filter(Gaussian blur) to prevent aliasing, since bicubic interpolation alone does not address frequency domain issues.

Performance

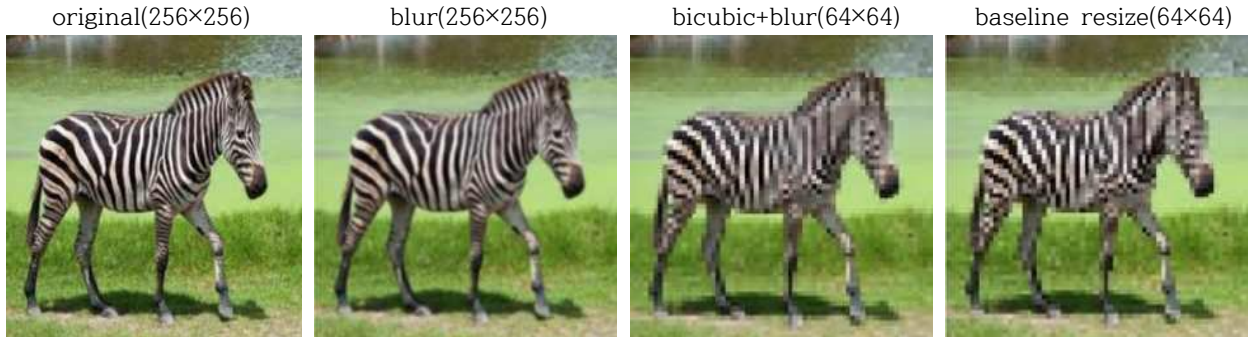


Figure 10. Anti-Aliasing Effects Using Gaussian Blur & Bicubic interpolation During Downsampling

Epochs	ACC(%) on Binary(64×64)		
	Baseline	step1+step2	step1+step2+Blur+Bicubic
10	61.05	62.21	62.27
20	66.03	66.69	66.80
30	66.41	68.52	69.18
40	66.15	69.11	70.11
50	65.73	70.07	70.34
Max	66.41	70.07	70.34

Table 10. Classification Accuracy over Epochs with resizing methods

Applying Gaussian blur before bicubic downsampling significantly reduces aliasing artifacts, as shown in Figure 3, where the zebra stripes remain smooth and consistent than baseline.

The classification accuracy in Table 3 confirms that Blur + Bicubic method outperforms all others, achieving the highest accuracy of 70.34%, suggesting that anti-aliasing preprocessing preserves more discriminative features.

6. STEP4. Binarization

※ Based on Step3

a) Otsu

Theoretical Motivation. Thresholding technique that aims to separate the foreground and background in a grayscale image by finding an optimal threshold value. The key motivation behind Otsu's approach is to maximize the between-class variance while minimizing the within-class variance, effectively ensuring the best possible separation between the two classes. The algorithm assumes that the image contains two classes of pixels following a bimodal histogram—typically corresponding to background and foreground.

Mechanism of Otsu. Evaluation all possible threshold values t in the image and selecting the one that maximizes the between class variance. The between class variance is computed as:

$$\sigma^2(t) = \omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2$$

$\omega_0(t), \omega_1(t)$ are the class probabilities,

$\mu_0(t), \mu_1(t)$ are the class means.

b) Adaptive thresholding(AdaTh)

Theoretical Motivation. Traditional global thresholding techniques(Otsu) assume a uniform illumination across the image, applying a single threshold value to binarize the entire image. However, real-world images often exhibit non-uniform lighting, shadows, or local contrast variations—particularly in datasets containing natural scenes, scanned documents, or biological samples.

Adaptive Thresholding (AdaTh) addresses this limitation by computing the threshold for each pixel based on the local neighborhood statistics, such as the mean or weighted mean (e.g., Gaussian-weighted) within a surrounding window. This enables more robust binarization in the presence of spatial lighting variations.

Mechanism of AdaTh. The binarized value $B(x, y)$ at pixel location (x, y) is defined as:

$$B(x, y) = \begin{cases} 255 & \text{if } I(x, y) > T(x, y) \\ 0 & \text{otherwise} \end{cases}$$

where $T(x, y)$ is a threshold compound using a function over the local neighborhood $N(x, y)$:

$$T(x, y) = \frac{1}{|N|} \sum_{(i, j) \in N} I(i, j) - C$$

or, in the Gaussian variant:

$$T(x, y) = \sum_{(i, j) \in N} G(i, j) \cdot I(i, j) - C$$

Performance

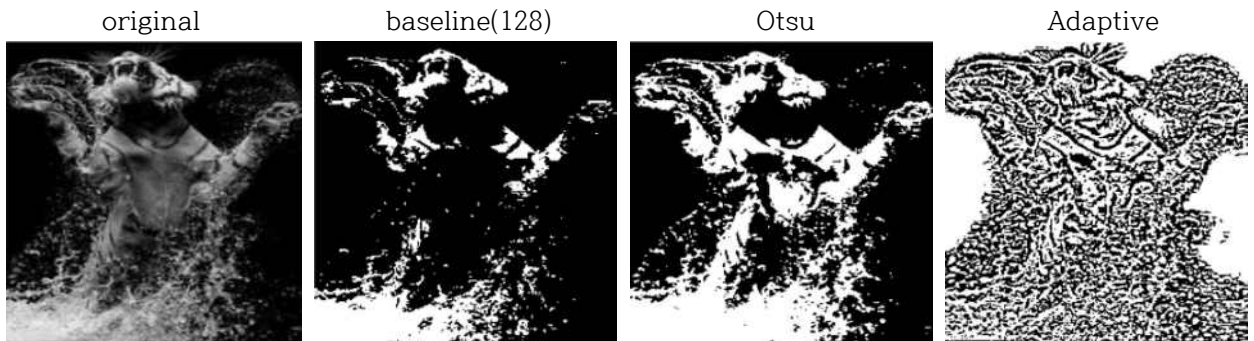


Figure 11. Comparison of Binarization Techniques

Epochs	ACC(%) on Binary(64×64)			
	Baseline	step1~3	step1~3+Otsu	step1~3+AdaTh
10	61.05	62.27	69.68	58.03
20	66.03	66.80	70.39	68.45
30	66.41	69.18	70.79	69.22
40	66.15	70.11	71.05	71.01
50	65.73	70.34	74.47	70.44
Max	66.41	70.34	74.47	71.01

Table 11. Classification Accuracy over Epochs with Binarization methods

Table 11 shows that both Otsu and adaptive thresholding methods improved classification performance. Notably, Otsu’s method achieved the highest accuracy of 74.47%, significantly outperforming the baseline.

In Step 2, classification is performed using edge images. The superior performance of Otsu thresholding in this context suggests a strong compatibility between edge-based representations and Otsu’s binarization approach. This is because Otsu’s method is designed to maximize inter-class variance, making it particularly effective when the image consists of clearly separable regions—such as object edges versus the background.

Edge images inherently exhibit strong intensity contrasts between object boundaries and their surroundings, often resulting in bimodal histograms, which aligns well with the assumptions of Otsu’s method. Consequently, Otsu can effectively emphasize meaningful structures while suppressing noise, leading to improved classification outcomes.

7. Final Results and Analysis

Method	Epochs					MAX
	10	20	30	40	50	
Upper bound(RGB)	79.34	84.49	88.97	87.95	86.88	88.97
Baseline	61.05	66.03	66.41	66.15	65.73	66.41
step1+Otsu	67.54	67.78	69.98	67.76	62.50	69.98
step1+AdaTh	68.42	68.89	71.07	69.34	69.34	71.07
step1+LBP	62.12	66.17	67.23	68.64	67.33	68.64
step1+LBP+Otsu	67.97	68.17	70.11	70.01	69.58	70.11
step1+LBP+AdaTh	68.76	69.47	71.42	70.99	70.98	71.42
step1~2	62.21	66.69	68.52	69.11	70.07	70.07
step1~3	62.27	66.80	69.18	70.11	70.34	70.34
step1~3+AdaTh	58.03	68.45	69.22	71.01	70.44	71.01
<i>Ours(step1~4)</i>	<i>69.68</i>	<i>70.39</i>	<i>70.79</i>	<i>71.05</i>	<i>74.47</i>	<i>74.47</i>

Table 12. ACC(%) on Binary(64×64) images, **Ours: crop2+LoG&Sobel+blur+Otsu**

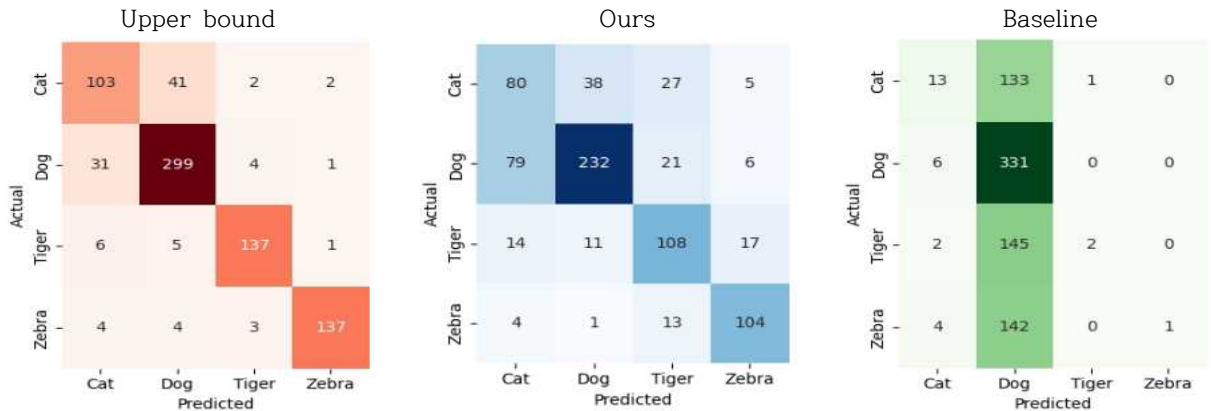


Figure 12. Confusion matrix, epoch=50

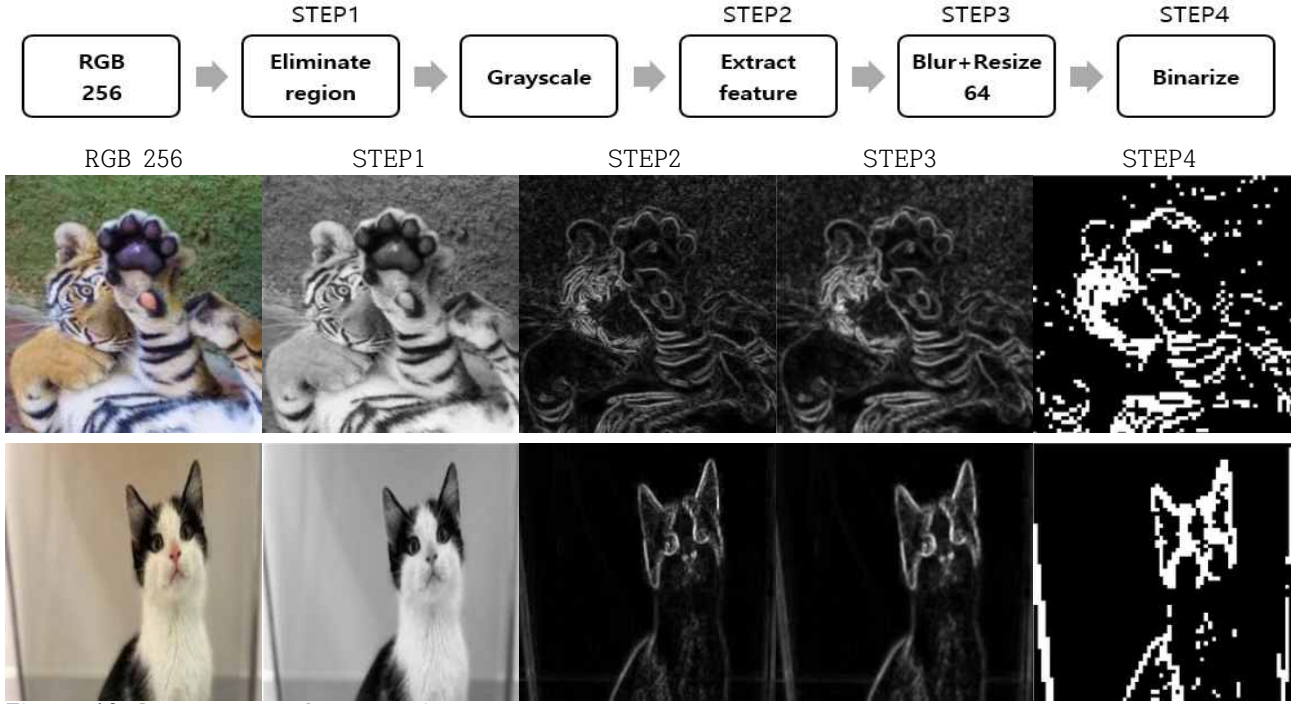


Figure 13. Images at each processing stage

Overall Performance Comparison. Our proposed method (Ours (step1~4)), which integrates cropping, Laplacian of Gaussian (LoG) and Sobel filtering, Gaussian blur, and Otsu thresholding, achieves the highest accuracy of 74.47% at epoch 50, outperforming the baseline by 8.32%. As shown in Table 12, this performance is consistent across epochs and exceeds that of all other individual or partial combinations. The result demonstrates the strong cumulative effect of combining edge enhancement and thresholding in binary image classification. The performance gap between our method and the RGB upper bound (83.97%) is narrowed significantly, despite the use of binary images only.

Effectiveness of Individual Enhancements. Each individual preprocessing step contributes positively to performance. For example, adding Otsu thresholding alone (step1+Otsu) yields 69.98%, improving over the baseline by 3.83%. Similarly, adaptive thresholding (step1+AdaTh) achieves 71.07%, the highest among all single-strategy enhancements. These results suggest that adaptive methods are more robust to intensity variation in binary images. Using texture descriptors like Local Binary Patterns (step1+LBP) also shows improvement, and its combination with thresholding—whether Otsu (70.11%) or adaptive (71.42%)—produces better results, showing the value of combining edge and texture-based features.

Impact of Cumulative Stepwise Integration. Stepwise accumulation of enhancements leads to incremental gains. The model achieves 70.07% with step1~2 and 70.34% with step1~3. Adding adaptive thresholding on top of those steps (step1~3+AdaTh) results in 71.01%, validating the complementary nature of the steps. This trend confirms that integrating multiple specialized preprocessing techniques is more effective than relying on any single method alone.

Confusion Matrix Analysis. As shown in Figure 12, confusion matrices offer further insight into model behavior. The baseline model exhibits a strong bias toward predicting the “Dog” class, misclassifying many “Cat” and “Zebra” samples. In contrast, our proposed method shows more balanced predictions across all classes, with stronger diagonal dominance, particularly in the “Cat” and “Tiger” rows. This indicates improved inter-class separability and lower confusion between visually similar classes. The RGB upper bound still performs best overall, but our method significantly closes the gap while using only binary inputs and lightweight preprocessing.

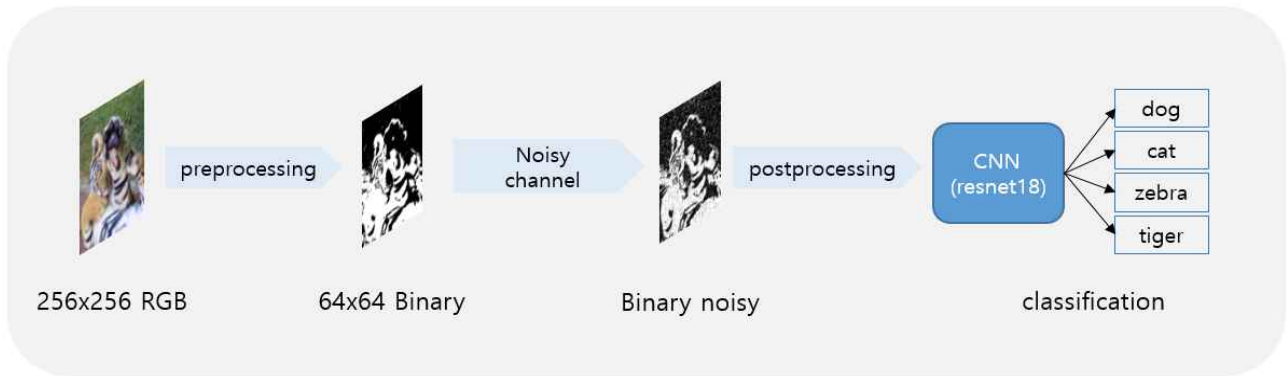


Figure 13. Overview of problem 2

Problem 2: Image preprocessing for classification, Part 2.

Assume a transmission channel introduces noise, causing pixel values in the received binary image to invert (i.e., 0 becomes 1, and 1 becomes 0). Your task is to design and build a classification system that maximizes performance under these conditions.

- 1) Let $p\%$ pixels are inverted by channel transmission. Check the performance degradation of the classification system. problem 2.1
- 2) Build your postprocessing(including denoising) method to maximize the performance. problem 2.2
- 3) As an alternative to image binarization for addressing bandwidth constraints, you can reduce the size of the image dimension to meet the bandwidth constraint. For example, transmitting $32 \times 32 \times 4$ -bit image is a possibility. Develop your own method to maximize the performance. problem 2.3

Problem 2.1

Epochs	Noise Ratio							
	Baseline				With Our Preprocessing			
	Upper	10%	25%	50%	Upper	10%	25%	50%
10	61.05	55.53	48.95	18.68	69.68	61.05	54.08	19.61
20	66.03	60.00	54.21	18.68	70.39	70.18	67.89	21.45
30	66.41	63.82	61.32	36.58	70.79	70.71	66.05	39.87
40	66.15	62.24	57.63	42.89	71.05	70.26	66.05	32.36
50	65.73	61.84	60.13	32.89	74.47	70.92	64.87	21.45
MAX	66.41	63.82	61.32	42.89	74.47	70.92	67.89	39.87

Table 13. ACC(%) on Binary(64×64)

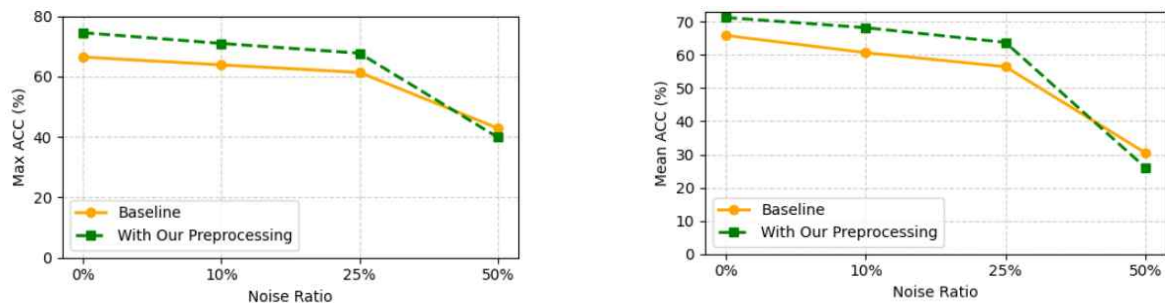


Figure 14. (left) Max ACC. (right) Mean ACC

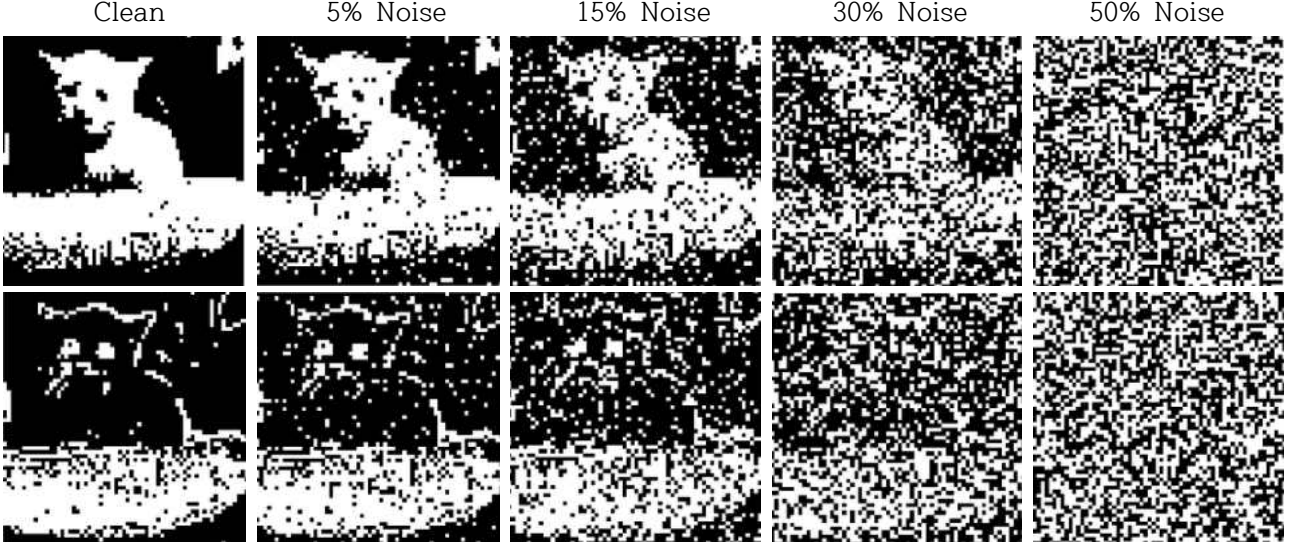


Figure 15. (Top) Add noise on baseline, (Bottom) apply our preprocessing before noising

Using Random Noise. To enhance the generalization capability of resnet-18 classifiers, it is essential to introduce random noise during training. Randomly applied noise simulates real-world degradations, encourages robustness to pixel-level perturbations, and prevents overfitting by forcing the model to focus on structure-preserving features rather than noise-sensitive artifacts.

Quantitative Evaluation under Noise. We evaluate the classification accuracy under varying levels of synthetic noise (0%-50%) using binary (64×64) images, comparing a baseline pipeline with our proposed preprocessing method applied prior to noise injection. As shown in Table 13 and Figure 14, our preprocessing consistently improves performance across all noise conditions except at the extreme 50% level. In terms of maximum accuracy, the proposed method outperforms the baseline by 8.06%, 7.10%, and 6.37% at 0%, 10%, and 25% noise levels, respectively. Likewise, the mean accuracy across epochs increases by 5.41%, 7.53%, and 7.34% in the same settings. Performance degrades under 50% noise for both methods.

Summary. Regardless of whether preprocessing is applied, the presence of noise itself has a significant impact on the model’s ability to distinguish between images. In particular, when the noise level reaches 50%, the classification accuracy drops drastically for both methods, effectively rendering the model incapable of meaningful prediction. This highlights the inherent vulnerability of binary image classification models to extreme noise contamination.

Problem 2.2

a) Conventional methods

1. **Median Filter.** Non-linear spatial filter. Replacing the value of a target pixel with the median value of its neighboring pixels within a window. Formally, let $N(x, y)$ denote the neighborhood centered at pixel (x, y) , the output $I'(x, y)$ is given by:

$$I'(x, y) = \text{median}(\{I(i, j) \mid (i, j) \in N(x, y)\})$$

Epochs	Median Filter					
	Baseline			With Our Preprocessing		
	$\epsilon=5\%$	$\epsilon=10\%$	$\epsilon=25\%$	$\epsilon=5\%$	$\epsilon=10\%$	$\epsilon=25\%$
10	56.32(+1.06)	57.63(+2.10)	52.37(+3.42)	60.12(-3.83)	60.97(-0.08)	57.50(+3.42)
20	61.05(-2.51)	60.42(+0.42)	54.55(+0.34)	68.16(-3.16)	68.55(-1.63)	65.53(-2.36)
30	64.53(-1.13)	62.32(-1.50)	61.79(+0.47)	71.05(-2.77)	69.61(-1.10)	67.63(+1.58)
40	62.89(-1.06)	63.34(+1.20)	57.63(+0.00)	70.13(-1.45)	69.74(-0.52)	66.58(+0.53)
50	62.16(-1.26)	61.03(+0.90)	61.11(+0.98)	70.79(-0.13)	71.05(-0.13)	65.53(+0.66)
MAX	64.53	63.34	61.32	71.05	71.05	67.63

Table 14. ACC(%) with Median filter. Parentheses indicate change from pre-denoising.

2. Component Filter. Basis of connected components analysis within the local neighborhood of a binary image. It identifies and preserves the dominant connected structure while suppressing isolated or weakly connected pixel clusters that are likely to be noise. The filtering process involves computing the connected components within a local window and retaining the component to which the central pixel belongs only if its area exceeds a predefined threshold. This method is particularly useful for eliminating sparse impulse noise while maintaining meaningful object continuity. Conceptually, for a binary patch P centered at pixel p , the filter preserves p if and only if the connected component containing p has a cardinality greater than a threshold τ :

$$I'(p) = \begin{cases} I(p), & \text{if } |C(p)| \geq \tau \\ 0, & \text{otherwise} \end{cases}$$

where $C(p)$ is the connected component of p in P .

Epochs	Component Filter					
	Baseline			With Our Preprocessing		
	$\epsilon=5\%$	$\epsilon=10\%$	$\epsilon=25\%$	$\epsilon=5\%$	$\epsilon=10\%$	$\epsilon=25\%$
10	49.87(-5.29)	48.16(-7.37)	44.08(-4.87)	63.16(-0.79)	58.68(-2.37)	45.92(-8.16)
20	61.45(-2.11)	61.05(+1.05)	55.00(+0.79)	65.66(-5.66)	65.66(-4.52)	56.58(-11.31)
30	59.74(-5.92)	60.13(+3.69)	58.55(-2.77)	65.39(-8.43)	63.29(-7.42)	53.42(-12.63)
40	64.47(+0.52)	61.84(-0.40)	56.84(-0.79)	67.63(-3.95)	65.79(-4.47)	59.61(-6.44)
50	62.50(+1.02)	60.92(-0.92)	58.55(-1.58)	68.16(-2.76)	65.66(-5.26)	54.08(-10.79)
MAX	64.47	61.84	58.55	68.16	65.79	59.61

Table 15. ACC(%) with Component filter. Parentheses indicate change from pre-denoising.

3. Majority Filter. Voting-based nonlinear filter commonly applied in binary image restoration. Within a local neighborhood, the value of the central pixel is updated based on the majority value of its neighbors. If the number of foreground pixels exceeds that of background pixels, the central pixel is set to 1, and vice versa. This approach effectively removes isolated noise pixels that do not conform to the majority context, assuming spatial coherence of the true signal. Formally, given a binary neighborhood $N(x,y)$, the majority filter computes:

$$I'(x,y) = \begin{cases} 1, & \text{if } \sum_{(i,j) \in N(x,y)} I(i,j) > \frac{|N|}{2} \\ 0, & \text{otherwise} \end{cases}$$

where $|N|$ denotes the total number of pixels in the neighborhood.

Epochs	Majority Filter					
	Baseline			With Our Preprocessing		
	$\epsilon=5\%$	$\epsilon=10\%$	$\epsilon=25\%$	$\epsilon=5\%$	$\epsilon=10\%$	$\epsilon=25\%$
10	54.50(-0.66)	54.58(-0.95)	49.76(+0.81)	60.26(-3.69)	58.03(-3.02)	56.18(+2.1)
20	58.97(-4.59)	59.97(-0.03)	55.55(+1.34)	68.29(-3.03)	69.08(-1.10)	61.18(-6.71)
30	62.00(-3.66)	62.08(-1.74)	60.95(-0.37)	69.34(-4.48)	71.05(+0.34)	64.47(-1.58)
40	59.24(-4.71)	61.55(-0.69)	58.45(+0.82)	71.32(-0.26)	68.95(-1.31)	66.05(0.00)
50	62.39(-1.13)	63.00(+1.16)	60.16(+0.03)	72.50(+1.58)	71.97(+1.05)	63.68(-1.19)
MAX	62.39	63.00	60.95	72.50	71.97	66.05

Table 16. ACC(%) with Majority filter. Parentheses indicate change from pre-denoising.

b) Analysis and Limitations of Conventional Methods

Method	Baseline			With Our Preprocessing		
	$\epsilon=5\%$	$\epsilon=10\%$	$\epsilon=25\%$	$\epsilon=5\%$	$\epsilon=10\%$	$\epsilon=25\%$
Median	64.53(-1.13)	63.34(+1.20)	61.79(+0.47)	71.05(-2.77)	71.05(-0.13)	67.63(+1.58)
Component	64.47(+0.52)	61.84(-0.40)	58.55(-2.77)	68.16(-2.76)	65.79(-4.47)	59.61(-6.44)
Majority	62.39(-1.13)	63.00(+1.16)	60.95(-0.37)	72.50(+1.58)	71.97(+1.05)	66.05(0.00)

Table 17. Max ACC (%) per filter under varying noise levels.

Analysis. Overall, applying denoising filters led to improved classification accuracy. Among the unprocessed images (i.e., without preprocessing), the median filter achieved the highest performance. In contrast, for images preprocessed using our method, the majority filter yielded the best results.

Interestingly, in both cases, when only a small amount of noise was added (approximately 5%), denoising resulted in a slight drop in accuracy. This suggests that the filters may have over-denoised the image, thereby removing meaningful structural details along with the noise.

Limitations of Conventional Methods. As mentioned earlier, conventional denoising methods such as the median, component, and majority filters tend to incur substantial information loss. For example, the median filter replaces pixel values based solely on local rankings, while component filtering removes small connected regions indiscriminately, which may include thin or small but meaningful structures.

This issue becomes more pronounced in the case of our preprocessing method, where images are converted to edge representations. These edge images encode critical boundary information using sparse, high-frequency signals. Applying conventional denoising techniques to such images often leads to the destruction of salient edge structures, effectively erasing essential features required for downstream classification.

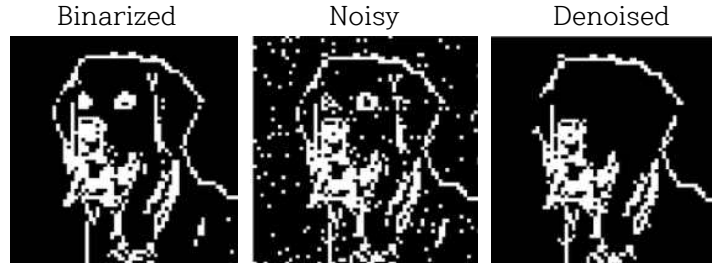


Figure 16. Denoising inevitably leads to information loss.

c) Ours

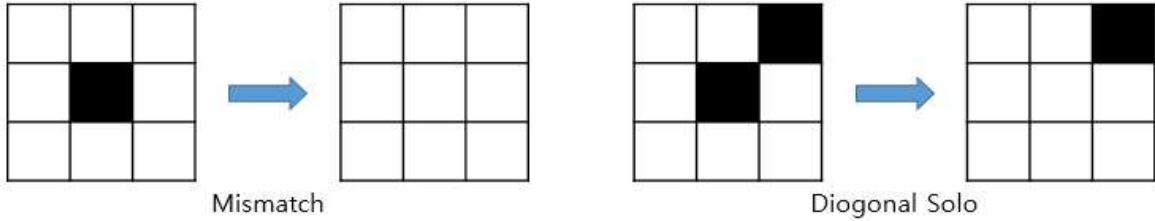


Figure 17. Ours methods

Theoretical Motivation. Conventional denoising filters often rely on uniform local heuristics (e.g., majority voting or median replacement), which do not adapt to the structural context of individual pixels. As a result, they tend to remove both noise and fine structural details indiscriminately—particularly problematic in edge-based binary images, where important features are sparse and localized.

To address this issue, we propose two structure-aware post-processing rules designed specifically for edge-enhanced binary representations: Strict Mismatch Rule and the Diagonal Solo Agreement Rule.

Mechanism.

Mismatch Rule is based on the intuition that a truly noisy pixel is often completely inconsistent with its surrounding context. It inverts the center pixel only when all neighboring pixels differ from it, thereby avoiding false positives in structured regions. This rule targets isolated impulse noise while minimizing overcorrection.

Diagonal Solo Rule is motivated by the geometric nature of edge continuity, especially in diagonal directions. It inverts the center pixel if exactly one diagonal neighbor matches its value, while all other 7 surrounding pixels disagree. This condition captures structural outliers that interrupt the natural diagonal flow of edges, particularly relevant in contour-preserving binary images.

Both rules are conservative and context-sensitive, aiming to remove noise only when structural inconsistency is strongly evident, thus preserving meaningful edge features while suppressing false patterns introduced by noise. By iteratively applying these two methods, we are able to denoise the

image while minimizing information loss.

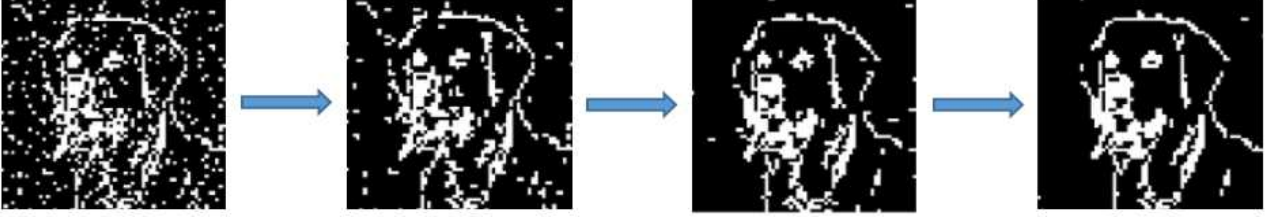


Figure 18. Image Denoising through Iterative Application

d) Performance

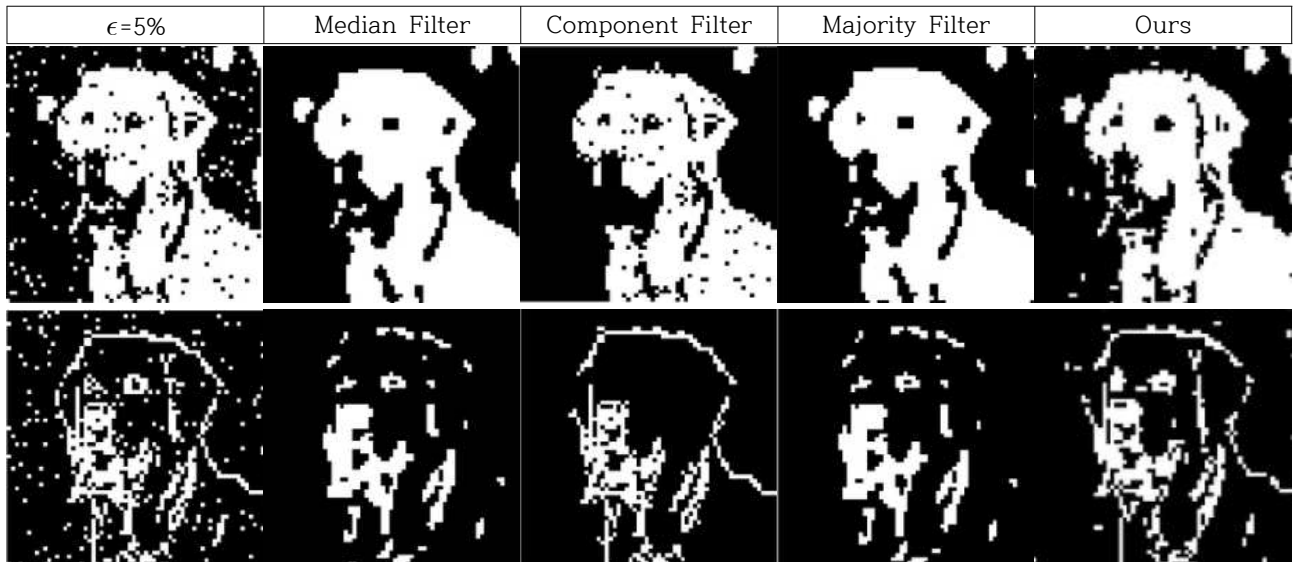
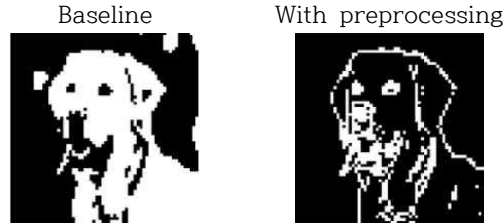
Epochs	Ours					
	Baseline			With Our Preprocessing		
	$\epsilon=5\%$	$\epsilon=10\%$	$\epsilon=25\%$	$\epsilon=5\%$	$\epsilon=10\%$	$\epsilon=25\%$
10	56.44(+1.18)	55.84(+0.31)	54.05(+5.10)	60.79(-3.16)	62.34(+1.29)	57.63(+3.55)
30	63.87(-1.79)	61.21(-2.61)	61.98(+0.66)	70.28(-3.54)	71.18(+0.47)	68.29(+2.24)
50	64.31(+0.89)	63.19(+1.35)	62.13(+2.00)	72.22(+1.30)	72.09(+1.17)	68.46(+3.59)
MAX	64.31	63.19	62.13	72.22	72.09	68.46

Table 18. ACC(%) with Ours. Parentheses indicate change from pre-denoising.

Method	Baseline			With Our Preprocessing		
	$\epsilon=5\%$	$\epsilon=10\%$	$\epsilon=25\%$	$\epsilon=5\%$	$\epsilon=10\%$	$\epsilon=25\%$
Median	64.53(-1.13)	63.34(+1.20)	61.79(+0.47)	71.05(-2.77)	71.05(-0.13)	67.63(+1.58)
Component	64.47(+0.52)	61.84(-0.40)	58.55(-2.77)	68.16(-2.76)	65.79(-4.47)	59.61(-6.44)
Majority	62.39(-1.13)	63.00(+1.16)	60.95(-0.37)	72.50(+1.58)	71.97(+1.05)	66.05(0.00)
Ours	64.31(+0.89)	63.19(+1.35)	62.13(+2.00)	72.22(+1.30)	72.09(+1.17)	68.46(+3.59)

Table 19. Max ACC (%) per filter under varying noise levels.

Analysis. Table 19 confirms the effectiveness of our preprocessing. Among all filtering strategies (Median, Component, Majority, and Ours), our method achieves higher maximum accuracy than most filters across various noise levels. Overall, these results confirm that the proposed preprocessing enhances both the training stability and the denoising performance, especially under severe noise, yielding the highest robustness and generalization.



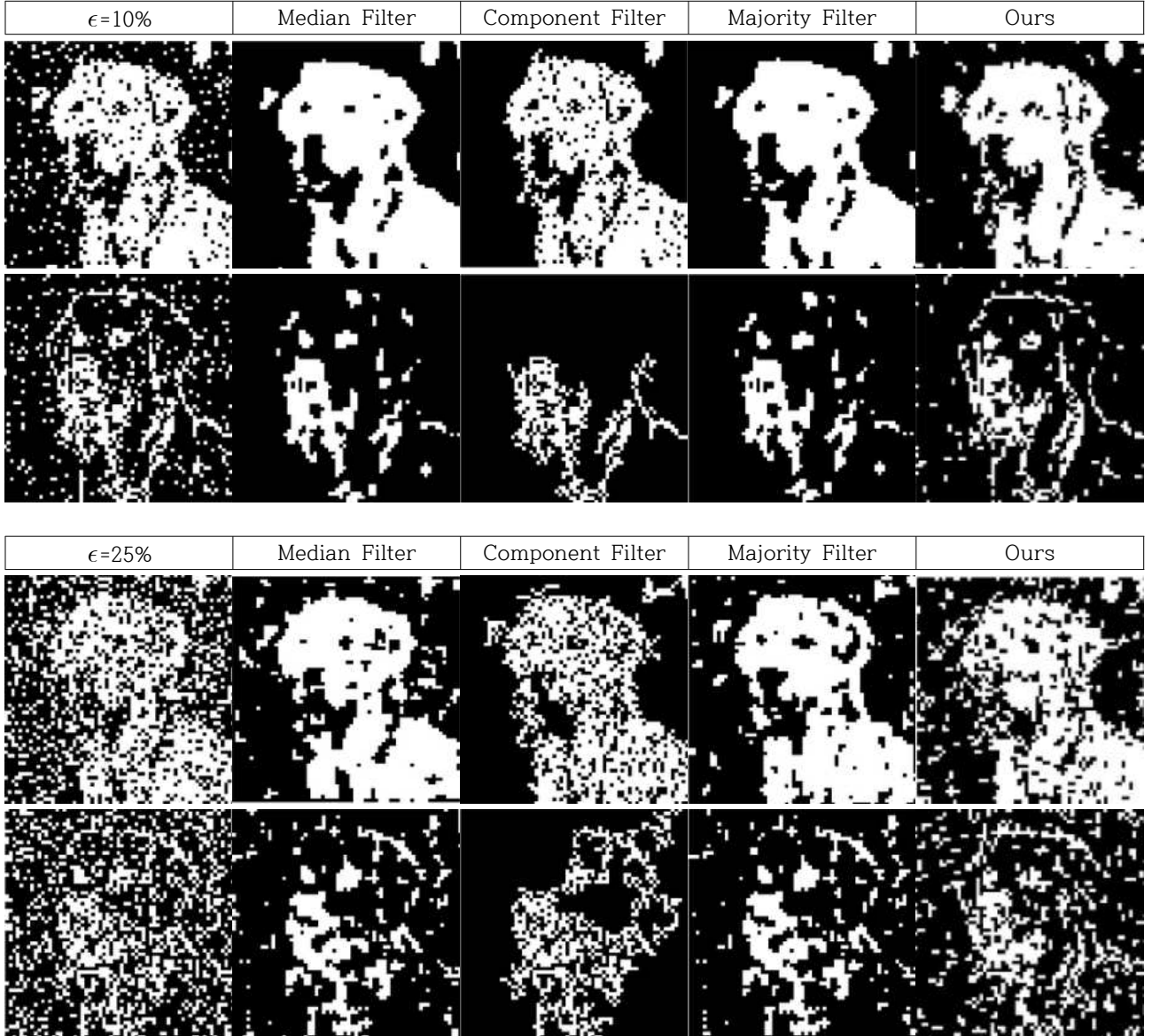


Figure 19. Comparison of Denoised Images Using Different Methods

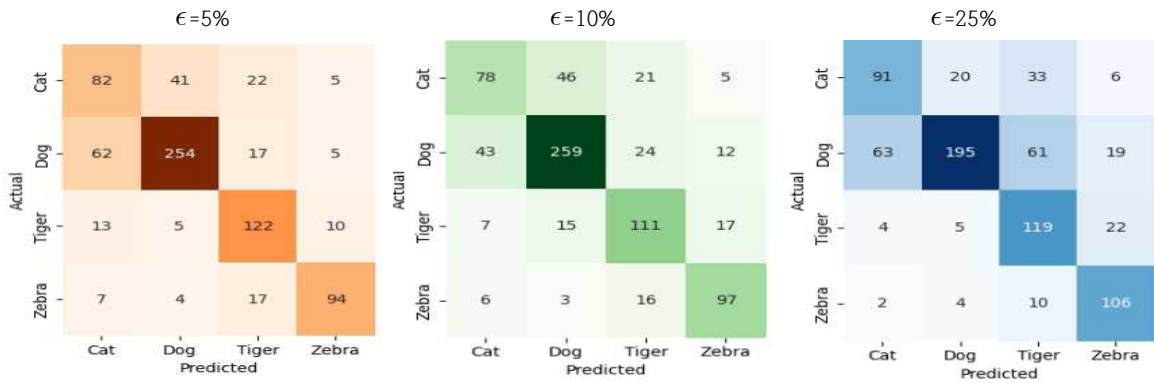


Figure 20. Confusion matrix, Our preprocessing + Our postprocessing

Limitation. As shown in Figure 19 and Table 19, applying our denoising method to images that have undergone our preprocessing clearly preserves object contours and achieves higher classification accuracy. In contrast, when the denoising method is applied without preprocessing, residual fine noise often remains, which interferes with accurate classification.

Problem 2.3

case 1. Noiseless Channel

a) Ours1: Two Channels Method

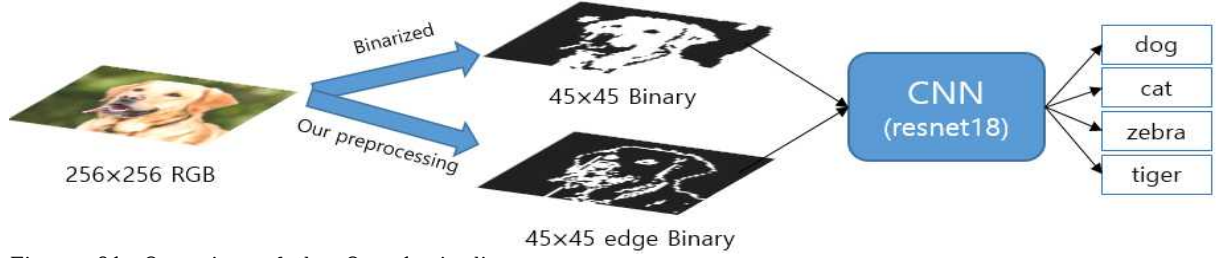
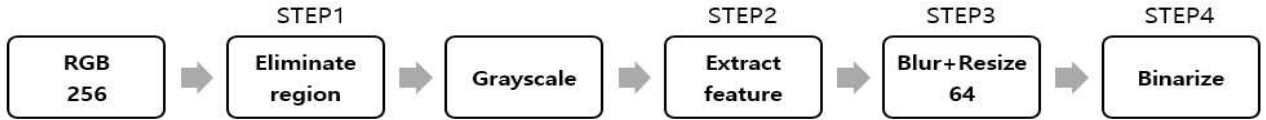


Figure 21. Overview of the Ours1 pipeline

Theoretical Motivation. The original 256×256 RGB image is processed through two parallel binary transformations: a standard binarization and a custom preprocessing method. Both producing 45×45 binary images that are jointly fed into a CNN (ResNet-18) for classification.

Under a strict transmission constraint of 64×64 bits (4096 bits total), a simple approach would be to downsample the image into a single 64×64 binary representation. However, this strategy may discard complementary or discriminative features crucial for accurate classification. Instead, by reducing the spatial resolution to 45×45 and employing two distinct preprocessing pipelines, we can construct a 2-channel input that remains within the same bit budget ($2 \times 45 \times 45 = 4050$ bits). This dual-channel strategy allows the network to access richer, more diverse information (such as fine-grained edges or structural cues) that a single binary image cannot convey alone. Consequently, this design balances bit-efficiency with representational diversity, leading to improved classification performance within the constrained bit environment.



Mechanism. To construct the dual-channel input, two binary images are generated from a single RGB image. One is produced using the baseline binarization method provided in the project, which omits step 2. The other is generated using our proposed four-step method described in Problem 1.3

Each binary image provides complementary information: one captures coarse object regions through standard binarization, while the other emphasizes structural boundaries via edge-based binarization. This dual representation enables the CNN to learn more discriminative features, thereby enhancing its understanding of the object during training.

Preprocessing pipelines for each input channel (Top: 1st channel, Bottom: 2nd channel)

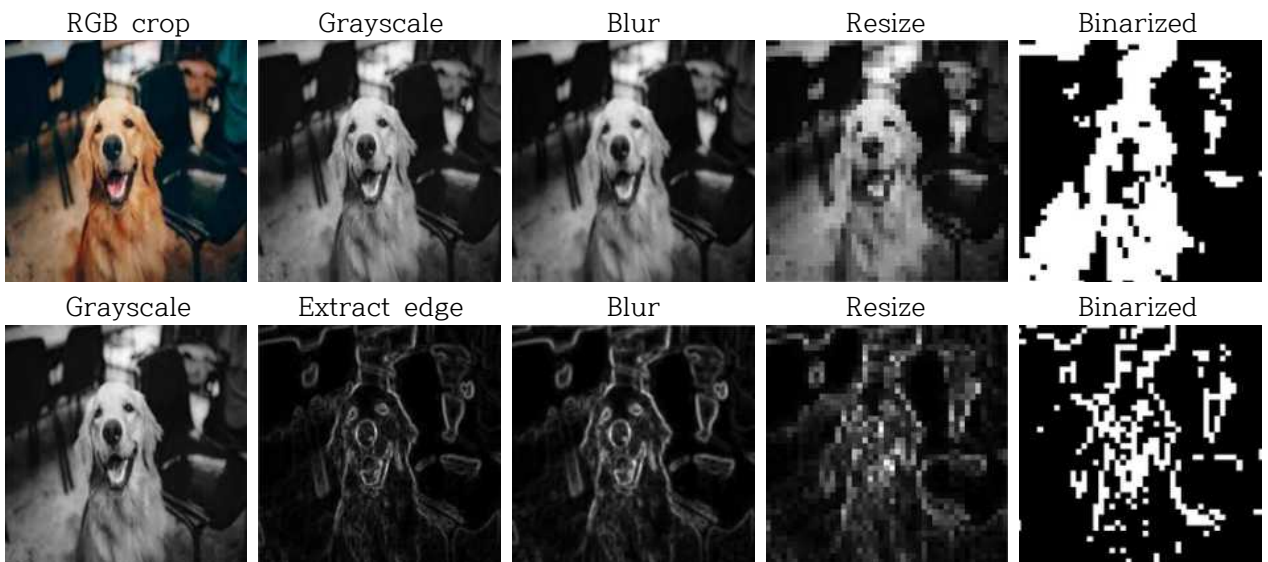


Figure 22. Image preprocessing for each channel

Performance

Method	Epochs					MAX
	10	20	30	40	50	
Upper bound(RGB)	79.34	84.49	88.97	87.95	86.88	88.97
Baseline	61.05	66.03	66.41	66.15	65.73	66.41
Ours in prob_1.3	69.68	70.39	70.79	71.05	74.47	74.47
Ours1	69.97	70.39	73.87	75.11	75.23	75.23

Table 20. ACC(%) on Binary(64×64) images

Analysis. Compared to training the CNN with a single 64×64 binary image, using two 45×45 binary images as a dual-channel input yields consistently better performance. This is because the two-channel approach provides complementary information: while the standard binarization channel captures the overall shape and object regions, the edge-based channel focuses on fine-grained structural boundaries. Despite the slight reduction in spatial resolution (from 64×64 to 45×45), the CNN benefits from the richer and more diverse representations across channels. The performance improvement observed across all epochs, highlights the effectiveness of leveraging dual-channel binary inputs for object classification.

2). Ours2: Image Decomposition Mehtod

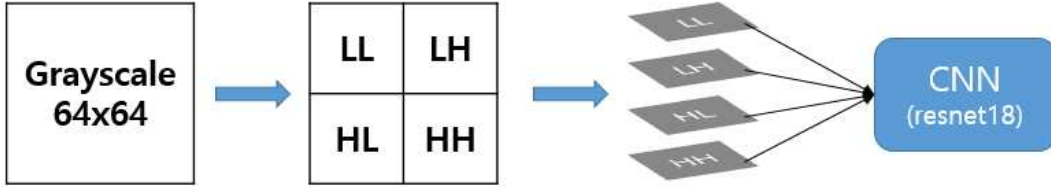


Figure 23. Overview of the Ours1 pipeline

Theoretical Motivation. To enable accurate image classification under strict bit constraints, we propose a compact multi-channel representation. Unlike traditional downsampled binary images that retain only coarse spatial structure, Ours2 decomposes an image into frequency subbands that separate low-frequency approximation from directional edge components. This decomposition allows us to preserve diverse and complementary information—such as object outlines (LL), horizontal and vertical edges (HL, LH), and fine textures (HH)—within a compact representation.

By selectively binarizing these subbands and packing them into a fixed 4096-bit budget (e.g., three 32×32 binary channels), we retain rich spatial and frequency-aware features that enhance CNN-based classification.

Mechanism. The input RGB image is first converted to grayscale and then decomposed using a single-level 2D DWT, yielding four frequency subbands: LL, LH, HL, and HH. Each subband is resized to 32×32 and binarized independently using Otsu’s thresholding method. The binarized maps are then concatenated channel-wise to form a multi-channel binary image input.

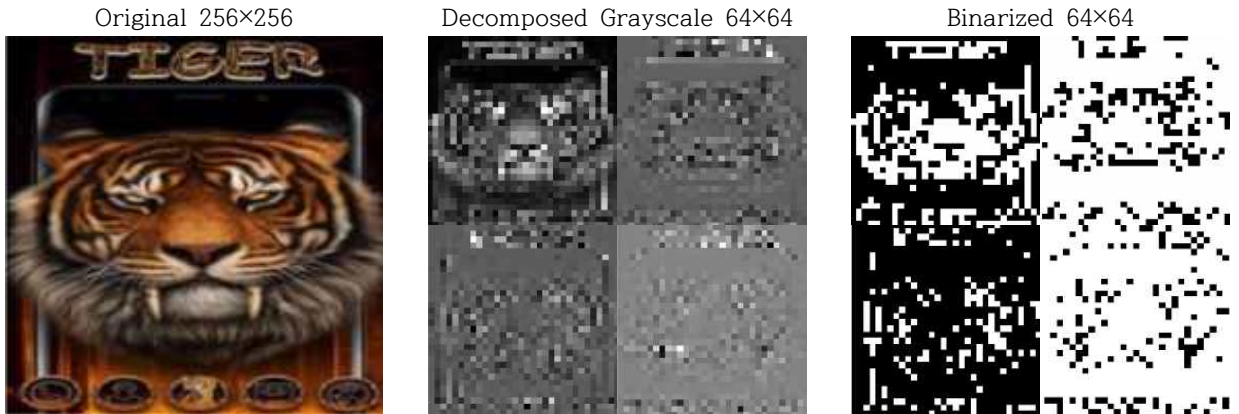


Figure 24. Ours2 Image preprocessing

Performance

Method	Epochs					MAX
	10	20	30	40	50	
Upper bound(RGB)	79.34	84.49	88.97	87.95	86.88	88.97
Baseline	61.05	66.03	66.41	66.15	65.73	66.41
Ours in prob_1.3	69.68	70.39	70.79	71.05	74.47	74.47
Ours1	69.97	70.39	73.87	75.11	75.23	75.23
Ours2	66.50	69.08	70.79	70.74	70.82	70.82

Table 21. ACC(%) on Binary(64×64) images

Analysis. Despite employing a 4-channel input constructed from binarized subbands (LL, LH, HL, HH), the overall classification performance was suboptimal. This suggests that although the wavelet transform decomposes the image into frequency-oriented components that may carry structural and texture cues, the binarization process potentially leads to significant information loss—especially in high-frequency bands like HL, LH, and HH.

Moreover, while the LL band preserves coarse shape and intensity structure, the remaining bands are highly sensitive to small variations, which may not generalize well during CNN training. The combination of binarization and downsampling might have weakened the semantic representation power of the input.

case 2. Noisy Channel

1) Ours3: Encoding and Decoding

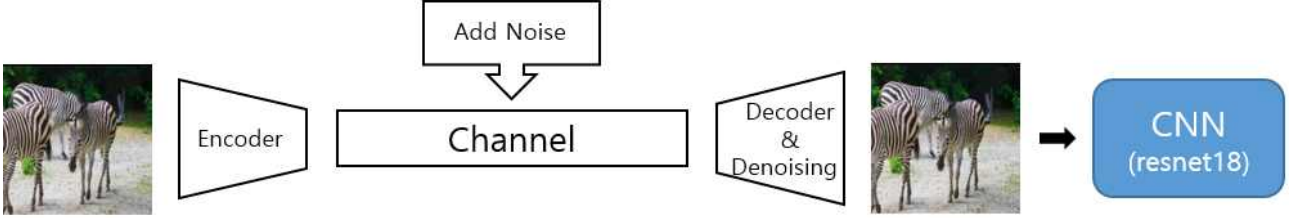


Figure 25. Overview of the Ours3 pipeline

Theoretical Motivation. According to the project constraint, the image data must be transmitted through a channel with a capacity of up to 2^{16} bits. This requirement implies that the total information to be sent, regardless of form, must remain within the specified bit limit. Instead of transmitting a directly downscaled or binarized version of the image, we adopt a more flexible approach that exploits standard image compression.

Mechanism. Specifically, we compress the original 256×256 RGB image using JPEG, reducing its size to fit within the given bit budget. Following this, we simulate transmission through a noisy channel by injecting controlled bit-level perturbations. To preserve classification performance, a lightweight denoising and decoding procedure is applied at the receiver side to restore a usable image. This pipeline leverages well-established compression and restoration mechanisms while adhering to strict channel capacity limitations, enabling the use of conventional convolutional neural networks without requiring architectural modifications.

Encoding: Adaptive JPEG Compression to Target Bit Budget. To meet the channel constraint of fixed transmission size, we adjust the JPEG compression quality to ensure the total image data fits within the target size. This is achieved through a binary search procedure over JPEG quality levels, ranging from 1 (lowest quality) to 95 (highest practical value for compression).

Let $Q \in [1, 95]$ denote the JPEG quality factor. For each candidate Q , we encode the image and measure the resulting compressed size S_Q . If $S_Q > \text{target_size}$, we reduce the quality; otherwise, we store the current result as the best candidate and increase the quality in search of higher fidelity within the constraint. Formally, this iterative search converges to the highest quality level such that $S_Q \leq \text{target_size}$. If the best candidate's size is strictly less than the target, we apply zero-padding to ensure that all output vectors maintain the same dimensionality. This normalization facilitates subsequent byte-level noise injection and decoding.

Channel Noise Injection. To simulate the effect of transmission errors through a noisy digital communication channel, we introduce random bit-level perturbations on the compressed JPEG data. Specifically, given a bit error rate p , each byte in the JPEG byte stream is randomly flipped with probability p . The flipping is implemented using the bitwise XOR operation with a randomly sampled integer from $\{1, \dots, 255\}$, ensuring non-trivial corruption across multiple bit positions. This simulates channel-level errors that may occur during wireless or hardware-constrained transmission environments.

Denoising. Following the injection of byte-wise noise into the JPEG-compressed bitstream, a simple denoising strategy is employed to enhance the recoverability of the corrupted data. Specifically, a 1D mean filter with a kernel size of 3 is applied to the noisy byte array. Each byte is replaced with the average of its local neighborhood:

$$\text{denoisedbyte}_i = \frac{1}{3}(b_{i-1} + b_i + b_{i+1})$$

This process smooths abrupt distortions introduced by random bit-flips without requiring knowledge of the JPEG format structure. While primitive, this method offers a lightweight and format-agnostic approach to mitigating noise effects at the byte level.

Decoding. The denoised byte sequence is interpreted as a JPEG image using *PIL.Image.open*. If the corrupted stream remains decodable, the image is recovered and converted to RGB format.

Performance

Method	Epochs					MAX
	10	20	30	40	50	
Upper bound(RGB)	79.34	84.49	88.97	87.95	86.88	88.97
Baseline	61.05	66.03	66.41	66.15	65.73	66.41
Ours in prob_1.3	69.68	70.39	70.79	71.05	74.47	74.47
Ours1	69.97	70.39	73.87	75.11	75.23	75.23
Ours2	66.50	69.08	70.79	70.74	70.82	70.82
Ours3	76.61	79.74	83.13	83.58	84.24	84.24

Table 22. ACC(%) on Binary(64×64) images

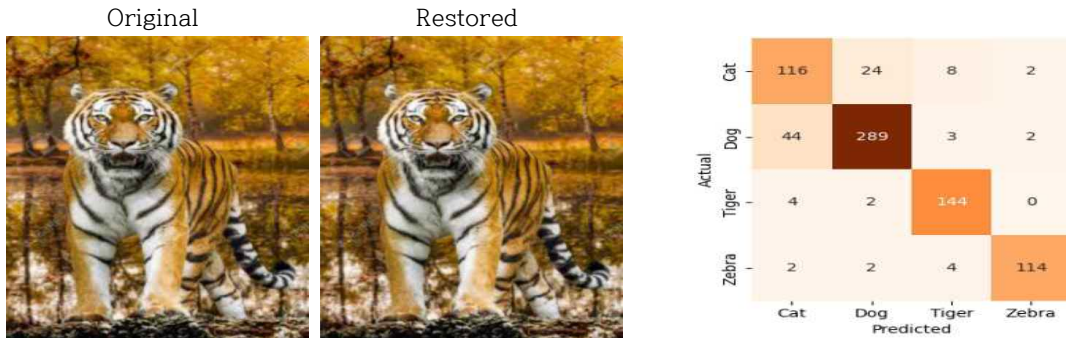


Figure 26. Ours3 performance, confusion matrix at epoch 50

Analysis. Ours3 demonstrates the highest performance, achieving 84.24% accuracy at epoch 50, outperforming all binary baselines and even surpassing earlier proposed methods such as Ours in prob.1.3 and Ours2. This result indicates that the JPEG-based encoding strategy not only preserves discriminative features but also provides robustness against compression and transmission artifacts.

Figure 26 illustrates visual examples of Ours3 preprocessing. Despite heavy JPEG compression and simulated channel corruption, the restored images retain content. These results validate the theoretical motivation behind Ours3: lossy compression with bit-level noise injection, when paired with simple post-transmission denoising, enables efficient and effective learning within strict bandwidth limits.